Java 24 features

Java 24 New Features With Examples

How to enable Preview Features of Java 24?

To compile and run code that contains preview features, we must specify additional command-line options. Since we are going to discuss some of the preview features of Java 24, we need to enable them explicitly during compilation and runtime:

Compilation:

```
javac --enable-preview --release 24 YourClass.java
```

Execution:

```
java --enable-preview YourClass
```

Let's start exploring Java 24 New Features with examples one by one.

Primitive Types in Patterns, instanceof, and switch (JEP-488, Second Preview)

State in Java 22:

Till Java 22, pattern matching was primarily applied to reference types. Developers could use pattern matching with the 'instanceof' operator to simplify type checks and casting for objects. However, primitive types were not included that limits the expressiveness and consistency of pattern matching across all data types.

*Example: Using **instanceof** with Primitive Types in Java 22*

```
Object value = 42;

if (value instanceof Integer i) {

    System.out.println("Integer value: " + i);

}
```

*Example: Using **switch** with Primitive Type Patterns in Java 22*

```
Object value = 3.14;

String result = switch (value) {
```

```
    case Integer i -> "Integer: " + i;
    case Double d -> "Double: " + d;
    default -> "Unknown type";
};
System.out.println(result);
```

Enhancements in Java 24:

First previewed in Java SE 23, this feature is re-previewed for this release. It is unchanged between Java SE 23 and this release. Java 24 expands pattern matching to include primitive types in all pattern contexts, such as the instanceof operator and switch expressions and statements. This enhancement allows for more concise and readable code when working with both object and primitive types. The instanceof operator and switch expressions and statements are extended to work with all primitive types.

Example: Using* instanceof *with Primitive Types

```
int value = 42;
if (value instanceof int) {    // Possible with Java 24
   System.out.println("Integer value: " + i);
}


int value = 10;
if (value instanceof int i && i > 5) {   // Possible with Java 24
  System.out.println("Value is an integer greater than 5");
}
```

Example: Using* switch *with Primitive Type Patterns

```
int status = 2;
String message = switch (status) {
    case 0 -> "OK";
    case 1 -> "Warning";
    case 2 -> "Error";
    case int i -> "Unknown status: " + i;
};
System.out.println(message);


double value = 3.14;
switch (value) {
  case int i -> System.out.println("value is an integer");
  case double d -> System.out.println("value is a double"); // Not possible before Java 24
  default -> System.out.println("value is something else");
}


double d = 3.14;
switch (d) {
  case 3.14 -> System.out.println("Value is PI");
  case 2.71 -> System.out.println("Value is Euler's number");
```

```
  default -> System.out.println("Value is not a recognized constant");
}

float rating = 0.0f;
switch (rating) {
  case 0f -> System.out.println("0 stars");
  case 2.5f -> System.out.println("Average");
  case 5f -> System.out.println("Best");
  default -> System.out.println("Invalid rating");
}
```

Flexible Constructor Bodies (JEP-492, Third Preview)

State in Java 23:

Prior to Java 22, the Java language required that any explicit constructor invocation (super() or this()) appear as the first statement in a constructor. This restriction meant that any initialization or validation code had to occur after these invocations, potentially leading to less intuitive code structures.

First previewed in Java 22 as JEP 447: Statements before super(…) (Preview) and previewed again in Java 23 as JEP 482: Flexible Constructor Bodies (Second Preview).

Enhancements in Java 24:

This feature is re-previewed for this release without any significant changes.

Example: Validating Arguments Before Calling Superclass Constructor

```
class Person {
   String name;
   Person(String name) {
      this.name = name;
   }
}

class Employee extends Person {
   Employee(String name) {
      if (name == null || name.isBlank()) {
         throw new IllegalArgumentException("Name cannot be empty");
      }
      super(name);
   }
}
```

Module Import Declarations (JEP-494, Second Preview)

This enhancement allows developers to import all public top-level classes and interfaces from the packages exported by a module using a single declaration, thereby simplifying the import process and reducing boilerplate code.

The module system required code to be organized into modules to take advantage of strong encapsulation and reliable configuration. However, importing modular libraries into non-modular codebases was inconvenient, often demanding workarounds or full migration to a modular system. Module Import Declarations enable you to succinctly import all of the packages exported by a module.

State in Java 23:

This feature was first previewed in Java 23.

Enhancements in Java 24:

This feature is re-previewed second time for this release. In this release:

- Type-import-on-demand declarations shadow module import declarations.
- Modules may declare a transitive dependence on the java.base module.
- The java.se module transitively requires the java.base module. Consequently, importing the java.se module imports the entire Java SE API.

This feature bridges the gap between modular and non-modular codebases, facilitating a smoother transition to modularity.

Java 24 refines module import declarations, improving compatibility and reducing potential conflicts when importing modules.

Syntax of Module Import Declaration:

import module moduleName;

Here, moduleName refers to the name of the module whose exported packages you wish to import.

How It Works?

A module import declaration imports all public top-level classes and interfaces from:

1. The packages exported by the specified module to the current module.
2. The packages exported by modules that are read by the current module due to reading the specified module. This means that if the specified module has transitive dependencies, their exported packages are also imported.

Examples:

1. **Importing the 'java.base' Module:**

The 'java.base' module is fundamental and exports numerous packages like 'java.util', 'java.io', etc. A module import declaration for java.base is equivalent to multiple on-demand package imports.

```
import module java.base;


public class BaseModuleExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        System.out.println("List created: " + list);
    }
}
```

In this example, classes List and ArrayList from the java.util package are accessible without individual import statements because java.util is exported by the java.base module.

1. **Importing the 'java.sql' Module:**

The 'java.sql' module exports packages like 'java.sql' and 'javax.sql'. You gain access to all public classes and interfaces in these packages by importing this module.

```
import module java.sql;


public class SqlModuleExample {
    public static void main(String[] args) throws SQLException {
        Connection conn = DriverManager.getConnection("jdbc:your_database_url");
        System.out.println("Connection established: " + conn);
    }
}
```

Here, Connection and DriverManager from the 'java.sql' package are accessible due to the module import declaration.

Handling Ambiguities:

When multiple modules export packages containing classes with the same simple name, ambiguities can arise. For example, both 'java.awt' and 'java.util' have a List class. To resolve such conflicts, we can use single-type import declarations or fully qualified class names.

```
import module java.desktop;

import module java.base;


public class AmbiguityExample {

   public static void main(String[] args) {

      java.util.List<String> list = new ArrayList<>();

      System.out.println("List created: " + list);

   }

}
```

In this scenario, specifying java.util.List clarifies which List class is being used.

Transitive dependency in Java Module System:

In Java's module system, **transitive dependency** means that when a module requires another module, it also gets access to the modules that the required module itself depends on.

- java.base is the fundamental module in Java. It contains core classes such as java.lang, java.util, java.io, etc. Every other module in Java implicitly depends on java.base, meaning it does not need to be explicitly required.
- java.se is an umbrella module that includes all the standard Java SE modules (e.g., java.sql, java.xml, java.desktop, etc.). The java.se module transitively requires java.base, meaning that when you depend on java.se, you automatically get access to java.base as well.

Example#1: Implicit Availability of java.base

Since java.base is required by all modules, we don't need to explicitly declare it in a module-info.java file.

```
// Java program using classes from java.base

public class BaseExample {

   public static void main(String[] args) {

      String message = "Hello, Java!";

      System.out.println(message.toUpperCase());

   }

}
```

This program runs without any module declaration because String and System.out are part of java.base.

Example#2: Explicitly Requiring java.se

If we create a modular application and require java.se, we automatically get access to java.base.

module-info.java

```
modulemy.application {

   requires java.se;

}
```

Main.java

```
import java.util.List;

import java.sql.Connection;


public class SEExample {

   public static void main(String[] args) {

      List<String> names = List.of("Java", "Angular", "Mongo DB");  // java.util (from java.base)

      System.out.println("Names: " + names);
```

```
        Connection conn = null;  // java.sql (from java.se)
        System.out.println("Database connection: " + conn);
    }
}
```

Since java.se is required, both java.util.List (from java.base) and java.sql.Connection (from java.sql) are available.

Example#3: Transitive Dependency

If a module A requires java.se, and another module B requires A, then B also gets access to java.base transitively.

module-info.java for moduleA

```
module moduleA {
    requires java.se;
}
```

module-info.java for moduleB

```
module moduleB {
    requires moduleA;  // No need to explicitly require java.base
}
```

Even though moduleB does not require java.base directly, it still gets access to classes from java.base because moduleA requires java.se, which transitively requires java.base.

Key Points:

1. java.base is always available and does not need to be explicitly required.
2. java.se is an umbrella module that requires multiple modules, including java.base.
3. When you require java.se, you automatically get access to java.base and all standard Java SE modules.
4. Transitive dependencies mean that if moduleA requires java.se, then moduleB (which requires moduleA) also gets access to java.base without explicitly requiring it.

Example: Importing various Modules

```
import module java.util;
import module java.base;
import module java.sql;
import java.util.Date;


public class ModuleImportConflictExample {
    public static void main(String[] args) {
        Date date = new Date();
        System.out.println("Date: " + date);
        List<String> list = new ArrayList<>();
        System.out.println("Module import works!");
    }
}
```

Stream Gatherers (JEP-485)

The Stream API provided a set of built-in intermediate operations (such as map, filter, and flatMap) for processing data streams. While powerful, these operations could be limiting when more complex transformations were required, often leading developers to write custom collectors or resort to less elegant solutions.

State in Java 23:

Stream Gatherers were proposed as a preview feature by JEP 461 in JDK 22 and re-previewed by JEP 473 in JDK 23.

Enhancements in Java 24:

This feature is finalized in JDK 24, without change. Java enhances the Stream API to support custom intermediate operations, known as Stream Gatherers. This feature allows developers to create more flexible and expressive stream pipelines, enabling transformations that were previously difficult or verbose to implement.

Example: Using a Custom Intermediate Operation

```
Stream<String> stream = Stream.of("a", "b", "c");

Stream<String> modifiedStream = stream.gather(

   () -> new StringBuilder(),

   (sb, s) -> sb.append(s.toUpperCase()),

   sb -> sb.toString()

);

modifiedStream.forEach(System.out::println);
```

Simple Source Files and Instance Main Methods (JEP 495, Fourth Preview)

This feature makes it easier for beginners to write their first programs without worrying about complex features meant for large applications. Instead of using a different version of Java, beginners can start with simple, single-class programs and gradually learn more advanced features as they improve.

Experienced developers can also benefit by writing small programs more quickly, without using extra structures meant for large-scale applications.

This feature was first introduced as a preview in Java 21 (JEP 445: Unnamed Classes and Instance Main Methods) and appeared again in Java 23 (JEP 477: Implicitly Declared Classes and Instance Main Methods). In Java 24, it is being previewed once more with new terminology and a revised title, but otherwise unchanged.

You may go through Simple Source Files and Instance Main Methods in Java Platform, Standard Edition Java Language Updates.

Class-File API (JEP-484)

Interacting with Java class files typically required third-party libraries or manual parsing, as there was no standard API for parsing, generating, or transforming class files. This lack of standardization could lead to compatibility issues and increased maintenance overhead.

State in Java 23:

The Class-File API was originally proposed as a preview feature by JEP 457 in JDK 22 and refined by JEP 466 in JDK 23.

Enhancements in Java 24:

The feature proposes to finalize the API in JDK 24 with minor changes, based on further experience and feedback.

A standard Class-File API provides a unified and reliable way to parse, generate, and transform Java class files. This API simplifies the development of tools and libraries that interact with bytecode, promoting consistency and reducing dependency on external libraries.

Example in Java 24:

```
import java.nio.file.Files;

import java.nio.file.Path;

import jdk.classfile.ClassFile;

import jdk.classfile.Method;


public class EnhancedClassFileAPIExample {

   public static void main(String[] args) throws Exception {

      Path path = Path.of("Example.class");

      byte[] classBytes = Files.readAllBytes(path);

      ClassFile classFile = ClassFile.read(classBytes);

      System.out.println("Class Name: " + classFile.thisClass());

      for (Method method : classFile.methods()) {

         System.out.println("Method: " + method.name());
```

```
    }
  }
}
```

Scoped Values (JEP-487, Fourth Preview)

State in Java 23:

In Java 23, scoped values were introduced as a third preview feature, enabling methods to share immutable data with their callees within a thread and with child threads. This mechanism offered a more straightforward and efficient alternative to thread-local variables, particularly when used with virtual threads and structured concurrency. However, the API included methods like callWhere and runWhere within the ScopedValue class, which, while functional, added complexity to the API's fluency.

The scoped values API was proposed for incubation by JEP 429 (JDK 20), proposed for preview by JEP 446 (JDK 21), and subsequently improved and refined by JEP 464 (JDK 22) and JEP 481 (JDK 23).

The feature is being proposed to re-preview the API once more in JDK 24 in order to gain additional experience and feedback, with one further change:

- callWhere and runWhere methods are removed from the ScopedValue class, leaving the API completely fluent. The only way to use one or more bound scoped values is via the ScopedValue.Carrier.call and ScopedValue.Carrier.run methods.

Enhancements in Java 24:

In Java 24, scoped values have reached their fourth preview, with significant refinements to improve the API's fluency and usability. The primary enhancement is the removal of the callWhere and runWhere methods from the ScopedValue class. Instead, the API now relies on the ScopedValue. Carrier.call and ScopedValue.Carrier.run methods for binding scoped values within a specific execution context. This change streamlines the API, making it more intuitive and consistent for developers.

Example: Using Scoped Values in Java 24

```
import java.lang.ScopedValue;

import java.lang.ScopedValue.Carrier;


public class ScopedValueExample {

  // Define a ScopedValue

  private static final ScopedValue<String> USER_ID = ScopedValue.newInstance();


  public static void main(String[] args) {

    // Create a carrier for the scoped value

    Carrier carrier = ScopedValue.where(USER_ID, "user123");


    // Run a task within the scope of the carrier

    carrier.run(() -> {

      // Access the scoped value within the scope

      System.out.println("User ID: " + USER_ID.get());

      performTask();

    });

  }


  private static void performTask() {

    // Access the scoped value in a nested method

    System.out.println("Performing task for User ID: " + USER_ID.get());

  }

}
```

In this example, the ScopedValue USER_ID is defined and then bound to the value “user123” within a specific scope using the ScopedValue.where method, which returns a Carrier. The Carrier.run method executes the provided task within the context of the scoped value binding. Within this scope, any method can access the USER_ID value using USER_ID.get(), ensuring that the data is shared safely and efficiently across method calls and threads.

This enhancement in Java 24 simplifies the API, makes it more fluent and easier to use, thereby improves the developer experience when working with scoped values.

Vector API (JEP-489, Ninth Incubator)

The Vector API allows developers to express vector computations that compile at runtime to optimal vector instructions on supported CPU architectures, achieving performance superior to equivalent scalar computations.

Example: Vector Addition

```
VectorSpecies<Float> SPECIES = FloatVector.SPECIES_PREFERRED;

float[] a = {1.0f, 2.0f, 3.0f, 4.0f};

float[] b = {5.0f, 6.0f, 7.0f, 8.0f};

float[] c = new float[4];

for (int i = 0; i < a.length; i += SPECIES.length()) {

   var va = FloatVector.fromArray(SPECIES, a, i);

   var vb = FloatVector.fromArray(SPECIES, b, i);

   var vc = va.add(vb);

   vc.intoArray(c, i);

}
```

Structured Concurrency (JEP-499, Fourth Preview)

Structured concurrency simplifies concurrent programming by treating groups of related tasks running in different threads as a single unit of work, streamlining error handling and cancellation, improving reliability, and enhancing observability.

Example: Using Structured Concurrency

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {

   Future<String> user = scope.fork(() -> findUser());

   Future<Integer> order = scope.fork(() -> fetchOrder());

   scope.join();

   scope.throwIfFailed();

   System.out.println(user.resultNow() + " " + order.resultNow());

}
```

Key Derivation Function API (JEP-478, Preview)

An API for Key Derivation Functions (KDFs) is introduced, which are cryptographic algorithms for deriving additional keys from a secret key and other data, enhancing security in cryptographic applications.

Example: Using a Key Derivation Function

```
SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256");

KeySpec spec = new PBEKeySpec(password, salt, iterations, keyLength);

SecretKey key = factory.generateSecret(spec);
```

Quantum-Resistant Module-Lattice-Based Key Encapsulation Mechanism (JEP-496)

Java 24 introduces quantum-resistant cryptographic algorithms to enhance security, notably implementing the Module-Lattice-Based Key Encapsulation Mechanism (ML-KEM). This algorithm is designed to safeguard data against potential threats posed by future quantum computing advancements.

Understanding ML-KEM:

ML-KEM is a key encapsulation mechanism that allows two parties to securely establish a shared secret over an insecure communication channel. This shared secret can then be used with symmetric-key cryptographic algorithms for tasks such as encryption and authentication. ML-KEM is designed to be secure against potential quantum computing attacks and has been standardized by the National Institute of Standards and Technology (NIST) in FIPS 203.

Using ML-KEM in Java 24:

Java 24 provides implementations of the KeyPairGenerator, KEM, and KeyFactory APIs for ML-KEM, supporting parameter sets ML-KEM-512, ML-KEM-768, and ML-KEM-1024 as defined in FIPS 203. Here's how you can utilize ML-KEM in your Java applications:

Generating an ML-KEM Key Pair:

To begin, generate a key pair using the KeyPairGenerator for ML-KEM:

```
import java.security.KeyPair;

import java.security.KeyPairGenerator;

import java.security.spec.NamedParameterSpec;


public class MLKEMExample {

    public static void main(String[] args) throws Exception {

        // Initialize the KeyPairGenerator with the ML-KEM algorithm

        KeyPairGenerator keyPairGenerator = KeyPairGenerator.getInstance("ML-KEM");


        // Specify the parameter set; ML-KEM-512 is used here

        keyPairGenerator.initialize(new NamedParameterSpec("ML-KEM-512"));


        // Generate the key pair

        KeyPair keyPair = keyPairGenerator.generateKeyPair();


        System.out.println("ML-KEM Key Pair generated successfully.");

        // The public and private keys can now be used for encapsulation and decapsulation

    }

}
```

Encapsulating a Secret Key (Sender's Perspective):

The sender uses the recipient's public key to encapsulate a secret key:

```
import javax.crypto.KEM;

import javax.crypto.SecretKey;


// Assuming 'publicKey' is the recipient's ML-KEM public key

KEM kem = KEM.getInstance("ML-KEM");

KEM.Encapsulator encapsulator = kem.encapsulator(publicKey);


// Encapsulate the secret key

KEM.Encapsulated encapsulated = encapsulator.encapsulate();


// Retrieve the encapsulated message to send to the recipient

byte[] encapsulation = encapsulated.getEncapsulation();


// The sender's copy of the secret key
```

SecretKey secretKeySender = encapsulated.getKey();

System.out.println("Secret key encapsulated successfully.");

Decapsulating the Secret Key (Recipient's Perspective):

The recipient uses their private key to decapsulate the received encapsulated message:

import javax.crypto.KEM;

import javax.crypto.SecretKey;

// Assuming 'privateKey' is the recipient's ML-KEM private key

KEM kem = KEM.getInstance("ML-KEM");

KEM.Decapsulator decapsulator = kem.decapsulator(privateKey);

// Decapsulate the secret key using the received encapsulation

SecretKey secretKeyReceiver = decapsulator.decapsulate(encapsulation);

System.out.println("Secret key decapsulated successfully.");

Note: In this example, both the sender and receiver derive the same secret key, which can then be used for secure communication. The parameter set chosen (e.g., ML-KEM-512) determines the security strength and performance characteristics. Ensure that both parties agree on the parameter set to maintain compatibility.

By integrating ML-KEM, Java 24 enhances the platform's resilience against quantum computing threats, ensuring secure key exchange mechanisms for future-proof applications.

Quantum-Resistant Module-Lattice-Based Digital Signature Algorithm (JEP-497)

Java 24 introduces the Module-Lattice-Based Digital Signature Algorithm (ML-DSA), a quantum-resistant digital signature mechanism. Digital signatures authenticate data and detect unauthorized modifications. ML-DSA is designed to be secure against future quantum computing threats and has been standardized by NIST in FIPS 204.

Example: Using ML-DSA

KeyPairGenerator kpg = KeyPairGenerator.getInstance("ML-DSA");

KeyPair kp = kpg.generateKeyPair();

Signature sig = Signature.getInstance("ML-DSA");

sig.initSign(kp.getPrivate());

sig.update(data);

byte[] signature = sig.sign();

These features collectively enhance Java's performance, security, and usability, solidifying its position as a modern and versatile programming language.